

Report 2: Safe Reinforcement Generative AI for Quadrotor Attitude Control Under Uncertainty

What is this Project about?

This project aims to build a simulation environment for drone control under wind disturbances and later investigate control/RL methods in generated wind environments. Therefore, my recent work focused on three directions: setting up the simulation infrastructure, learning Unity and implementing an initial wind field, and studying AE/VAE models as preparation for future generative wind modeling.

Research Objectives for the Next Report

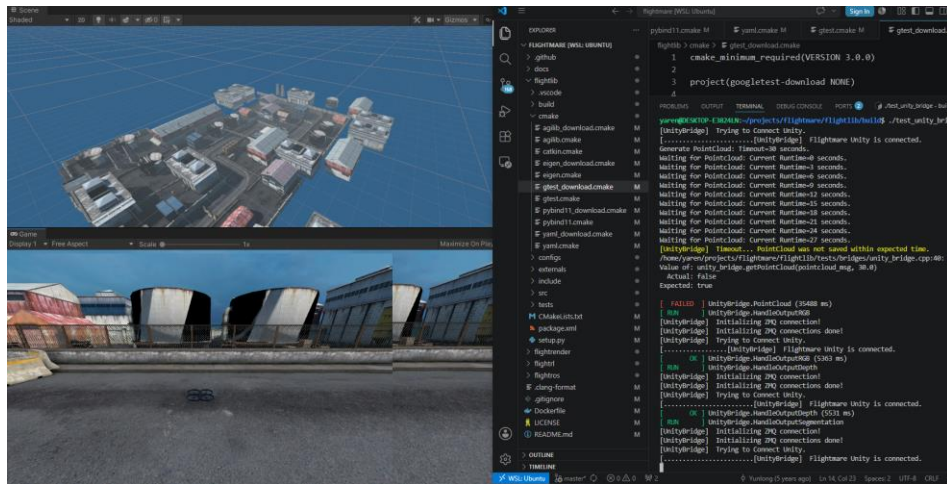
In the next stage, I plan to examine the CVAE structure on MNIST. Unlike VAE, CVAE adds label or condition information to the generation process. While this condition can be a numerical class in MNIST, in the future, different wind categories can be defined as conditions for wind simulation. This would allow the model to generate controlled scenarios according to specific wind types.

Contents

- 1) Flightmare–Unity Simulation Connection
- 2) Unity Learning and Initial Wind Simulation
- 3) AutoEncoder (AE)
- 4) Variational AutoEncoder (VAE)

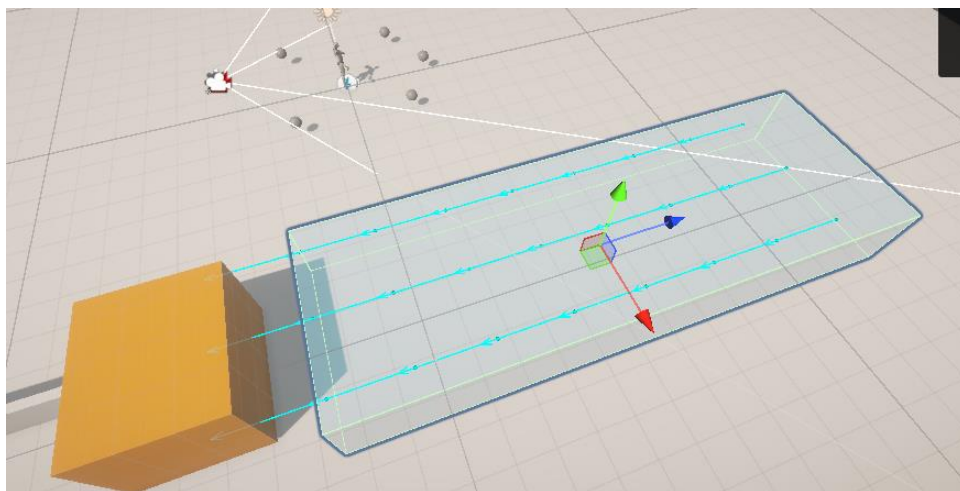
Flightmare–Unity Simulation Connection

At this stage, I worked on Flightmare[1] and Flightmare-Unity environments to establish the simulation infrastructure of the project. While preparing the Unity scene on the Flightmare-Unity side, I installed and compiled the Flightmare client side on WSL/Ubuntu and tested the connection between the Unity environment running on Windows and the Flightmare structure on Linux/WSL. During this process, I prepared the development environment using VS Code's WSL connection, and observed that the C++/Python-based structure on the Flightmare client side communicated with the Unity scene via UnityBridge. As a result of the test_unity_bridge experiment, the connection between Unity and the Flightmare client was successfully established, and the spawning of the drone in the Unity scene was tested. This work can be considered as the first successful step in the technical connection infrastructure necessary for future simulation experiments based on drone control, wind disturbances, and reinforcement learning.



Unity Learning and Initial Wind Simulation

Before starting to develop a wind simulation, I created several small projects and test scenes to better understand the Unity environment. During these studies, I tried to learn the basic components of Unity, such as GameObject, component, script structure, collider usage, camera settings, character movement, physics interactions, and basic game logic. Then, using this knowledge, I created the first simple environment for the wind simulation. In the Unity scene, I defined a unidirectional and constant-speed wind zone that is effective within a specific area. This structure can currently be thought of as a uniform vector field; that is, the direction and intensity are the same at every point in the wind zone. Although it is not yet a complex wind model, this work has provided a foundation for creating a more realistic wind zone that changes depending on location and time in the future. In the next stages, the goal is to define the wind with mathematical equations such as $Wind(x, y, z)$ or $Wind(x, y, z, t)$ depending on time, and thus develop a more complex wind disturbance environment with different directions and speeds in different regions. I also used the Gizmos feature in the script to visualize the wind direction in the wind zone; this method will allow the wind direction to be visually tracked in more complex wind environments.



AE

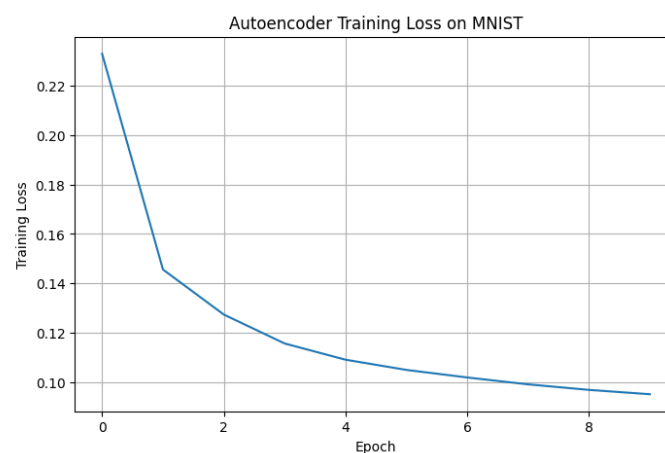
An autoencoder is a structure that compresses and reconstructs data using a neural network. Unlike traditional compression techniques, instead of storing the exact same data, it learns the key features of the data in a smaller-sized latent representation. Therefore, the compressed representation can be quite

small; however, some corruption may occur in the reconstructed data. The data passes through the encoder, latent space, and decoder stages in sequence. The encoder converts the data into a smaller representation, the latent space is the compressed version of the data, and the decoder attempts to convert this representation in latent space back to the original data size using the parameters learned during training.

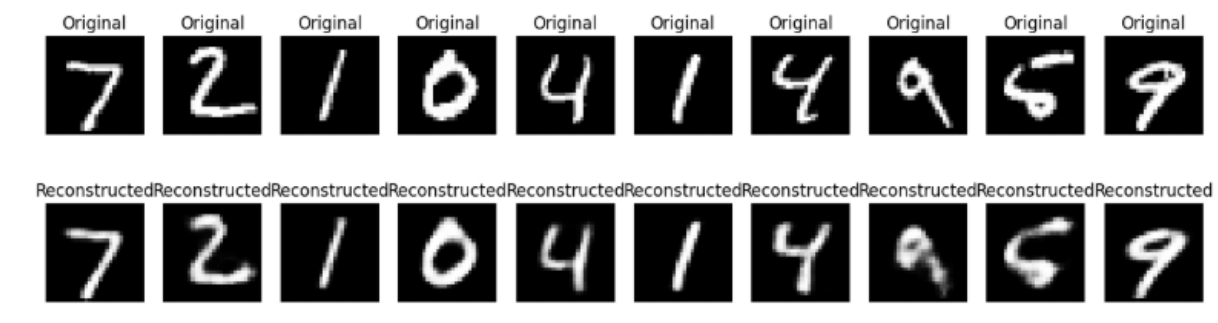
In the model I'm using, the encoder first converts the 28x28 MNIST image into a 784-dimensional vector, and then compresses it incrementally through several layers into a $784 \rightarrow 256 \rightarrow 64 \rightarrow 32$ -dimensional latent representation. The decoder performs the reverse process, expanding the 32-dimensional representation in latent space back to $64 \rightarrow 256 \rightarrow 784$ dimensions and converting it back to the 28x28 image format. The reason for incremental compression is to allow the model to learn the key features of the data gradually. During the forward pass, the data is passed through the encoder and decoder layers; the input image is first converted into a one-dimensional vector, and the decoder output is then converted back to the image size. The sigmoid function used at the end of the decoder limits the generated pixel values to a range of 0 to 1, making them suitable for a normalized image format.

The loss function and optimizer used in the model are fundamental parts of the training process. Loss is the value that measures the error made by the model. The optimizer is the algorithm that updates the model's weights to reduce this error. In an autoencoder model, the goal is for the output produced by the model to resemble the input data as closely as possible. Therefore, when calculating loss, the reconstructed image is compared with the original image. In this study, I used MSELoss (Mean Squared Error Loss). MSELoss squares the differences between the output and input and calculates the average of these values as the loss value. A small loss value indicates that the output produced by the model is close to the input data and that the reconstruction process was successful. A large loss value indicates that the model's output is further from the original data.

During the training cycle, the model learns the training data in iterations called epochs. In each epoch, the model sees the training data from beginning to end once and tries to reduce the reconstruction error by updating its weights. Looking at the training loss graph, it can be seen that the loss value decreases in each epoch. This indicates that the reconstructed output produced by the model is getting closer to the original input and that the autoencoder is learning to reconstruct the images more successfully.



In conclusion, the original and reconstructed images are as follows.



In conclusion, the model successfully compressed the MNIST images into a low-dimensional latent representation and then converted them back to image format. The reconstructed images generally resemble the original images in shape; however, some blurring and minor distortions are observed in certain details. This is because the autoencoder attempts to reconstruct the data by learning its basic structure, rather than creating an exact copy. The decrease in training loss over the epochs also indicates that the model's reconstruction performance improves over time.

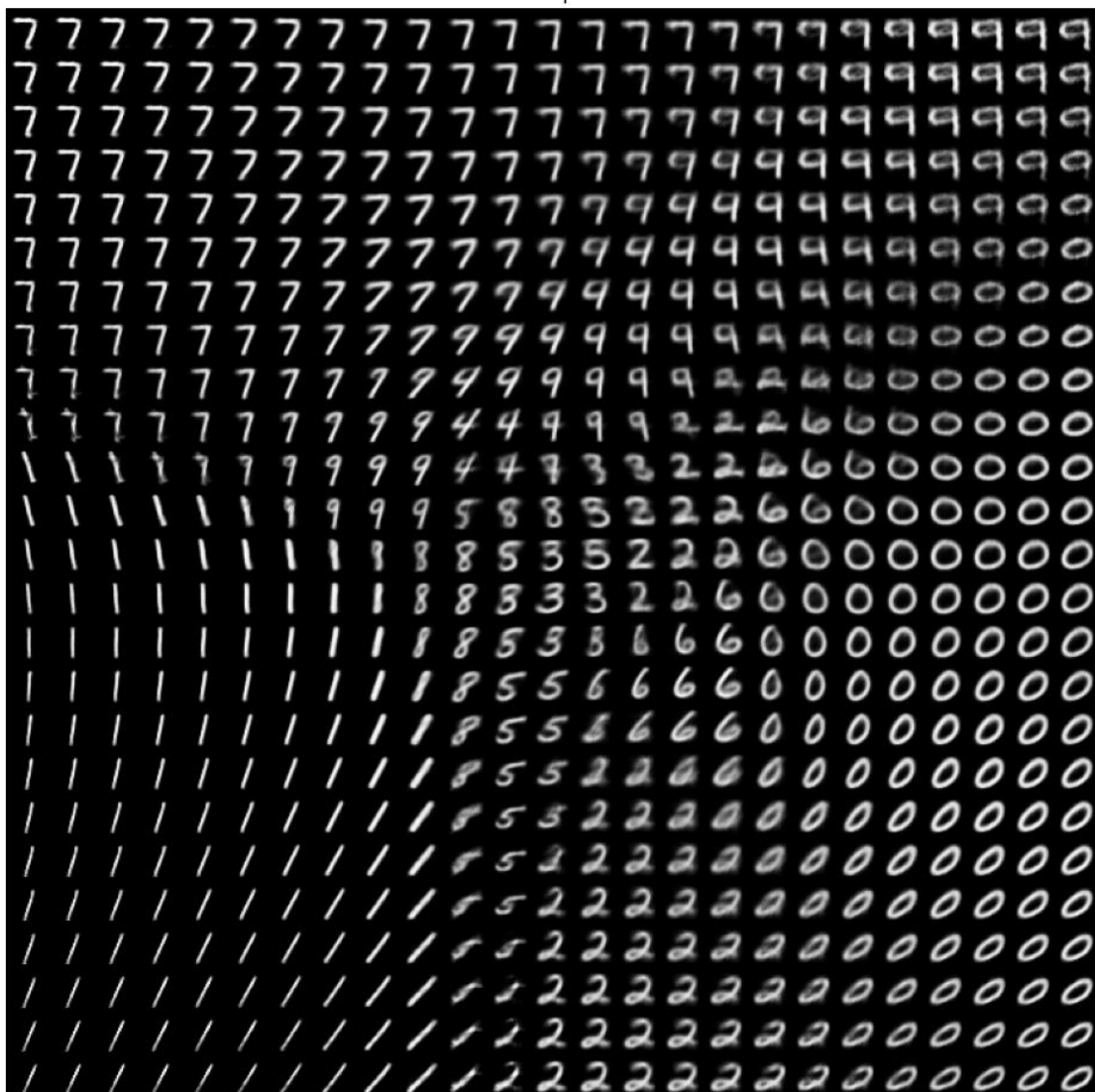
VAE

In this experiment, a Variational Autoencoder (VAE) was trained on the MNIST dataset. Unlike a standard autoencoder, the VAE encoder does not produce a single deterministic latent vector. Instead, it learns the parameters of a latent probability distribution, represented by mean and log-variance values. A latent vector is then sampled from this distribution using the reparameterization trick and passed to the decoder to reconstruct the input image. The loss function consists of two parts: reconstruction loss and KL divergence. Reconstruction loss encourages the generated output to be similar to the original input, while KL divergence regularizes the latent space to follow a standard normal distribution. After training, new digit-like images can be generated by sampling random latent vectors and passing them directly to the decoder.

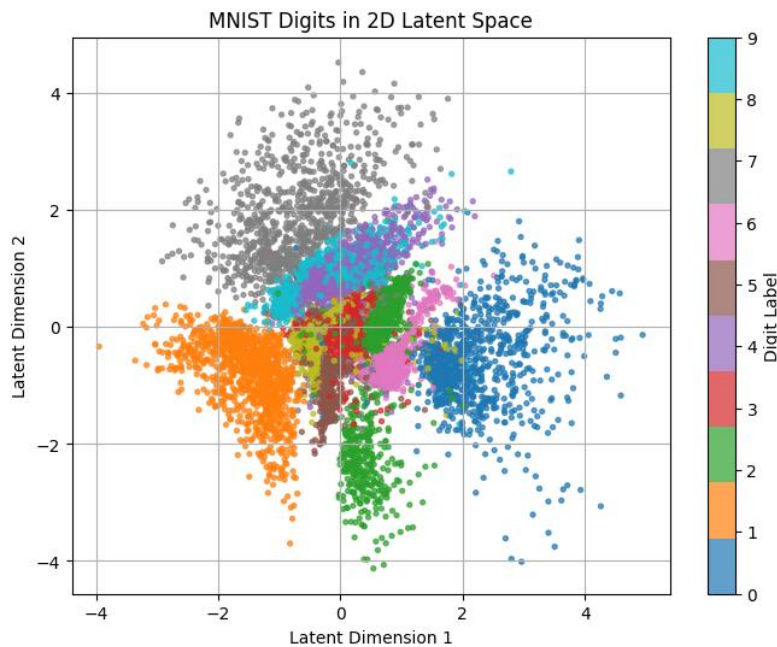
Generated New MNIST-like Images from Random Latent Vectors



2D VAE Latent Space Manifold

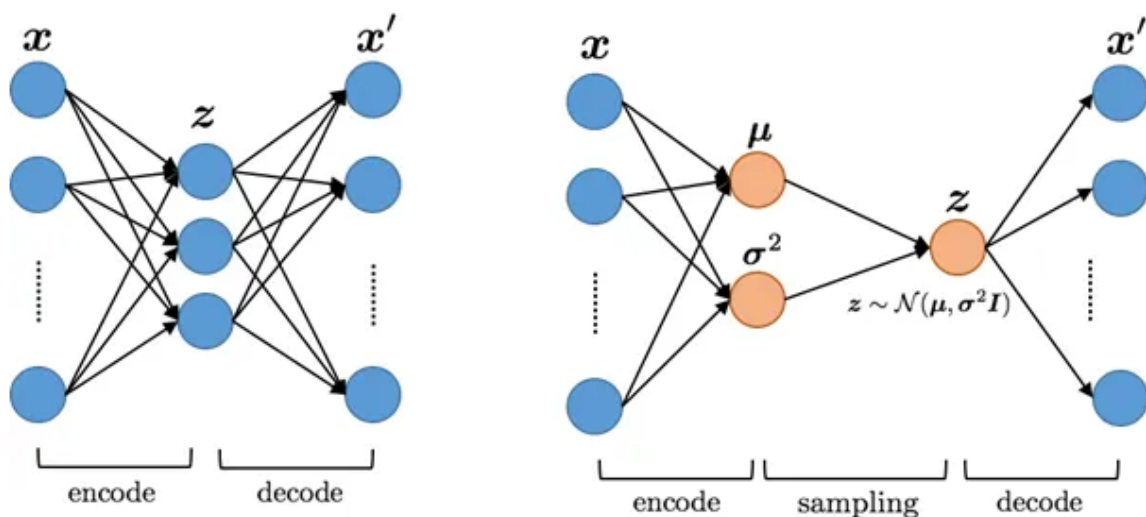


The 2D latent manifold visualization above was used to examine the structure of the latent space learned by VAE. Selected, regularly spaced grid points were passed through a decoder to generate new digit images. The fact that points close to each other produce similar outputs and that the digits change smoothly along the grid indicates that VAE has learned a regular and continuous latent representation.



The graph shows that visually similar digit structures are located close to each other in the learned latent space.

AE vs VAE



“In the figure above, we can see the difference between AE and VAE. In AE, the input (x) is compressed into a code (z) and converted back to its original form (x'). In VAE, mean (μ) and variance (σ^2) values are obtained from the input (x). Using these values, the code (z) is sampled. This

step is called the sampling step. Afterwards, this code is decompressed and converted back to its original form (x').”[2]

References

[1] <https://github.com/uzh-rpg/flightmare>

[2] <https://cihanogun.medium.com/variational-autoencoder-vae-nedir-autoencoderdan-ne-fark%C4%B1-var%C4%B1r-c3f44f6f25c8>